



PRAKTIKUM PROGRAMMIERUNG 2

Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Letzter Stand:
28.09.2025

Seite
1 von 7

1 Vorbereitung

- Machen Sie eine Ausarbeitung zu den Lösungen dieses Aufgabenblattes.
 - Die Ausarbeitung enthält zu jeder dokumentierten Teilaufgabe eine Überschrift mit der Nummer und dem Namen der Teilaufgabe.
 - Die Ausarbeitung enthält vorn ein von Ihrem Textprogramm generiertes Inhaltsverzeichnis.
 - Die Ausarbeitung enthält die Namen der Studierenden, die bei diesem Aufgabenblatt zusammengearbeitet haben (max. 2).
 - Bei allen Programmieraufgaben lassen Sie die zugehörige main()-Klasse einmal laufen und machen von der Ausgabe Ihres Programmes einen Screenshot, der in der Ausarbeitung dokumentiert wird.
- Erstellen Sie ein Verzeichnis/einen Workspace für alle Teilaufgaben dieses Aufgabenblattes
- Für die Teilaufgaben erstellen Sie Projekte, so wie es in den jeweiligen Teilaufgaben angegeben ist.

Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Seite
2 von 7

2 (10P) Fragen zu fortgeschrittenen Begriffen der OOP

2.1 (5P) Grundbegriffe

Erklären Sie stichpunktartig die folgenden Begriffe anhand von Beispielen:

- Serialisierbarkeit einer Klasse
- Serialisieren / Deserialisieren
- Package
- Annotation
- Generische Klasse

2.2 (5 P) Erklären Sie den Unterschied zwischen den 2 folgenden Klassendiagrammen

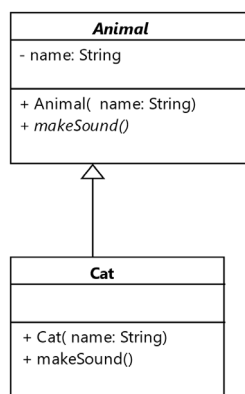


Diagramm (i)

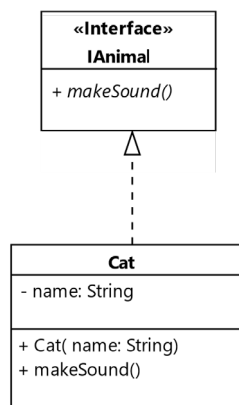


Diagramm (ii)

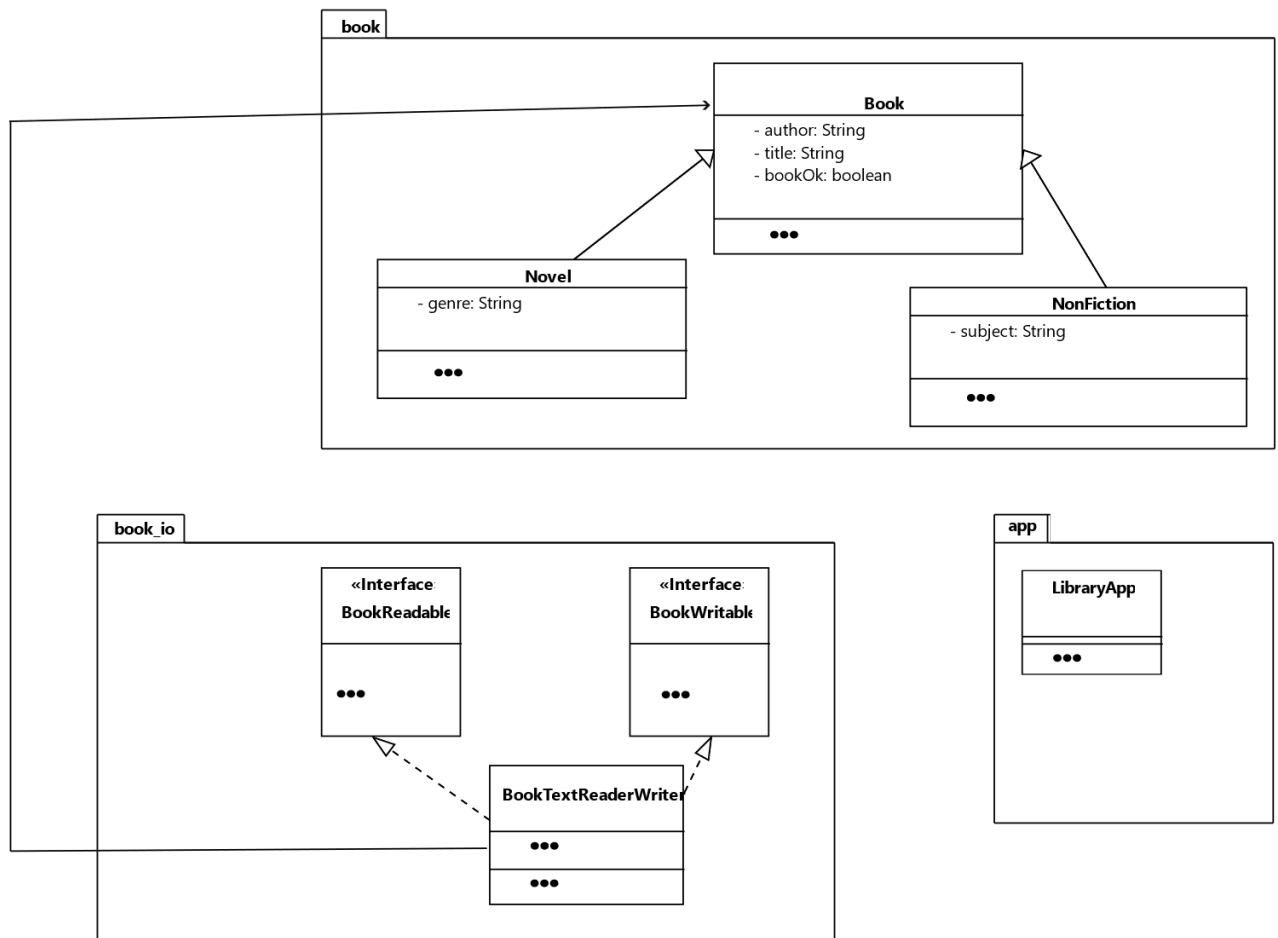
Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Seite
3 von 7

3 Packages

Nehmen Sie die Klassen und Interfaces Ihrer Book-Anwendung aus Aufgabenblatt 3 / Aufgabe 6 und kopieren Sie sie in ein neues Projekt **BookPackages**.

Verpacken Sie die Klassen Ihres Projektes nun in folgende Package-Aufteilung:



Details und Hinweise:

- Denken Sie daran: Klassen, die andere **Klassen aus anderen Packages benutzen**, müssen diese **importieren**!
- Dokumentieren Sie in ihrer Ausarbeitung nun:
 - In welcher Klasse stecken nun welche package-Anweisungen?
 - In welcher Klasse stecken nun welche import-Anweisungen?
 - Welche Verzeichnisstrukturen sind durch die Package-Aufteilung entstanden?



Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Seite
4 von 7

4 (20 P) Serialisierung / Deserialisierung

Sie schreiben ein größeres Softwarepaket für einen Hersteller von Schreib- und Zeichenstiften.

4.1 Klassenhierarchie serialisierbarer Klassen

Es gibt folgende Klassen für die Produkte des Stiftherstellers:

- Eine Klasse für die Grundeigenschaften von Stiften. Ein **Stift**
 - hat eine **Farbe** und einen **Hersteller** (beides String)
 - ist **serialisierbar**.
 - Hat public **getter** für ihre beiden Attribute.
- Eine Spezialisierung für **Bleistifte**. Sie hat keine zusätzlichen Eigenschaften und **erbt** nur die Eigenschaften von Stiften.
- Eine Spezialisierung für **Filzstifte**. Er hat ebenfalls keine zusätzlichen Eigenschaften und **erbt** nur die Eigenschaften von Stiften.

Details und Hinweise:

- Sie dürfen den drei Klassen deutsch- oder englischsprachige Namen geben, wie Sie möchten. Bleiben Sie aber bei einer Sprache für die drei Klassen.
- Alle drei Klassen haben
 - Defaultkonstruktor
 - Voll parametrisierten Konstruktor
 - Kopierkonstruktor
 - **toString()**-Methode
- Denken Sie in den Konstruktoren an die passenden **super(...)**-Aufrufe!
- Alle drei Klassen stecken im Package **products**.

Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Seite
5 von 7

4.2 Serialisierung der Klassenhierarchie aus 4.1: Teil 1

Schreiben Sie ein **Interface ProductReaderWriter**, welches die Methoden

```
public ArrayList<Stift> read();  
public void write(ArrayList<Stift> list);
```

vorgibt.

Details und Hinweise:

- Das Interface steckt im Package **product_io**
- Denken Sie daran: Klassen, die andere **Klassen oder Interfaces aus anderen Packages benutzen**, müssen diese **importieren**!

4.3 Serialisierung der Klassenhierarchie aus 4.1: Teil 2

Schreiben Sie eine Klasse **ProductFileReaderWriter**, die das Interface aus 4.2 implementiert. Sie enthält:

- Einen **Namen für die zu serialisierende** Datei.
- Einen **Namen für die zu deserialisierende** Datei.
- Einen voll parametrisierten Konstruktor, der beide Namen initialisiert.
- Die vom Interface **ProductReaderWriter** vorgegebenen Methoden.

Details und Hinweise:

- Die Methode **public void write(ArrayList<Stift> list)** soll die Stifte der übergebenen Liste mittels eines **ObjectOutputStream** auf Datei **serialisieren**. Als Dateiname wird der String aus dem dafür vorgesehenen Attribut der Klasse verwendet.
- Die Methode **public ArrayList<Stift> read()** soll aus einer Datei mit serialisierten Stiften diese wieder auslesen (**deserialisieren**). Als Dateiname wird der String aus dem dafür vorgesehenen Attribut der Klasse verwendet.
- Die Klasse steckt ebenfalls im Package **product_io**
- Denken Sie daran: Klassen, die andere **Klassen aus anderen Packages benutzen**, müssen diese **importieren**!
- Der folgende Ausschnitt einer **main()**-Methode zeigt die Anwendung der beiden Methoden:

```
ArrayList<Stift> list = new ArrayList<Stift>();  
ProductFileReaderWriter rw = new ProductFileReaderWriter("file.bin", "file.bin");  
list.add(new Stift("Red", "Drawy Inc."));  
list.add(new Bleistift("Dark Grey", "Coal Inc."));  
list.add(new FilzStift("Yellow", "Sunny Inc."));  
  
// Auf Datei schreiben/serialisieren  
rw.write(list);  
// Wieder auslesen/deserialisieren  
ArrayList<Stift> list2 = rw.read();
```



Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

Seite
6 von 7

4.4 Main()-Klasse ProductApp

Schreiben Sie eine main()-Klasse **ProductApp**, die einen ähnlichen Code wie in 4.3 ausführt. Sie verwenden dabei die von Ihnen gewählten Klassennamen für die Stift-Klassen. Die wieder deserialisierten Objekte geben Sie in einer geeigneten Schleife aus.

Details und Hinweise:

- Die Klasse steckt im Package **app**
- Denken Sie daran: Klassen, die andere **Klassen aus anderen Packages benutzen**, müssen diese **importieren!**
- Erstellen Sie zu dem Programm in Ihrer Ausarbeitung einen Screenshot, der den Lauf des Programms zeigt.
- Erstellen Sie in Ihrer Ausarbeitung ein **Klassendiagramm**, welches die **Beziehungen** zwischen den Klassen, Packages und Interface zeigt. Details innerhalb der Klassendarstellung sind nicht gefragt.

Aufgabenblock IV: Fortgeschrittene OOP: Packages, ObjectStreams und einfache Generics

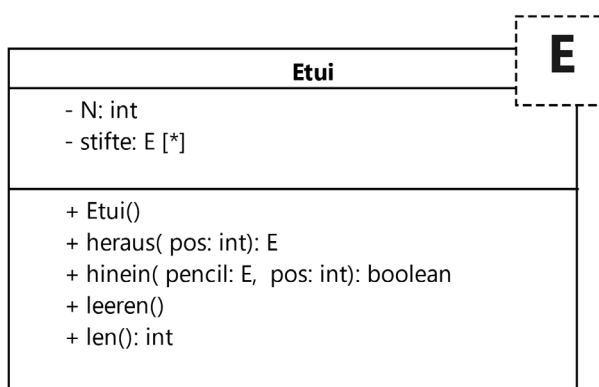
Seite
7 von 7

5 Erste einfache Generics

Nehmen Sie Ihre Klassen/Interface/Packages aus Aufgabe 4 und kopieren Sie diese in ein neues Projekt **FirstGenerics**.

Der Stifthersteller, für den Sie die Software entwickeln, möchte nun auch Stift-Etuis anbieten.

Schreiben Sie eine generische Klasse **Etui<E>** mit folgendem Aufbau:



Die Klasse enthält

- Eine **int**-Konstante **N=10** für die maximale Anzahl Stifte/Elemente im Etui.
- Ein generisches Array **stifte** vom Typ **E** (**keine ArrayList!!**)
- Einen Default-Konstruktor, der das Array **stifte** passend mit Platz für **N** Elemente erzeugt.
- Eine Methode **public E heraus(int pos)**, die das Element an Position **pos** zurückgibt.
 - Wenn **pos** ungültig ist oder leer (also **null**) ist, wird **null** zurückgegeben.
- Eine Methode **public boolean hinein(E stift, int pos)**, die ein Element vom Typ **E** übergeben bekommt.
 - Wenn die Position **pos** frei ist (also **null** ist), dann wird das Element auf diese Position gelegt und die Methode gibt **true** zurück.
 - Wenn die Position **pos** NICHT frei ist (also nicht **null** ist), dann wird einfach **false** zurückgegeben.
 - Wenn die Position **pos** ungültig ist, wird ebenfalls **false** zurückgegeben.
- Eine Methode **public void leeren()**, die das Etui leert. Die Referenzen im Array **stifte** werden dabei alle auf **null** gesetzt.
- Eine Methode **public int len()**, die die Länge des Arrays **stifte** zurückliefert.
-

Details und Hinweise:

- Die Klasse steckt im Package **products**
- Ergänzen Sie Ihre **main()**-Klasse **ProductApp** aus Aufgabenteil 4.4, so dass diese
 - Ein Etui erzeugt und mit mindestens 4 Stiften befüllt.
 - Auch die anderen Methoden des Etuis sollen mindestens einmal aufgerufen werden.
- Erstellen Sie zu dem Programm in Ihrer Ausarbeitung einen Screenshot, der den Lauf des Programms zeigt.